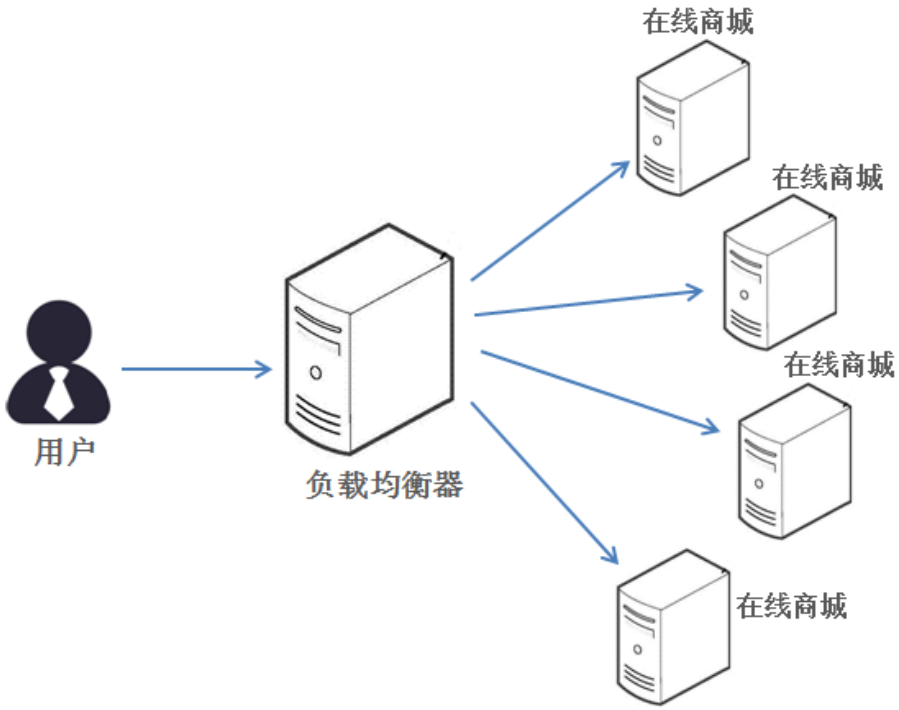
1.分布式与集群

1.1.集群

我们都知道，单机能够处理的并发请求数是有限的。那么，当单机处理到达瓶颈时，可以将单机复制几份，这样就构成了一个“集群”。集群中每台服务器就叫做这个集群的一个“节点”，所有节点构成了一个集群。每个节点都提供相同的服务，那么这样系统的处理能力就相当于提升了好几倍（有几个节点就相当于提升了这么多倍）。

但问题是用户的请求究竟由哪个节点来处理呢？最好能够让此时此刻负载较小的节点来处理，这样使得每个节点的负载压力都比较平均。要实现这个功能，就需要在所有节点之前增加一个“调度者”的角色，用户的所有请求都先交给它，然后它根据当前所有节点的负载情况，决定将这个请求交给哪个节点处理。这个“调度者”就是负载均衡服务器。

比如：下面是一个在线商城的服务集群架构图：

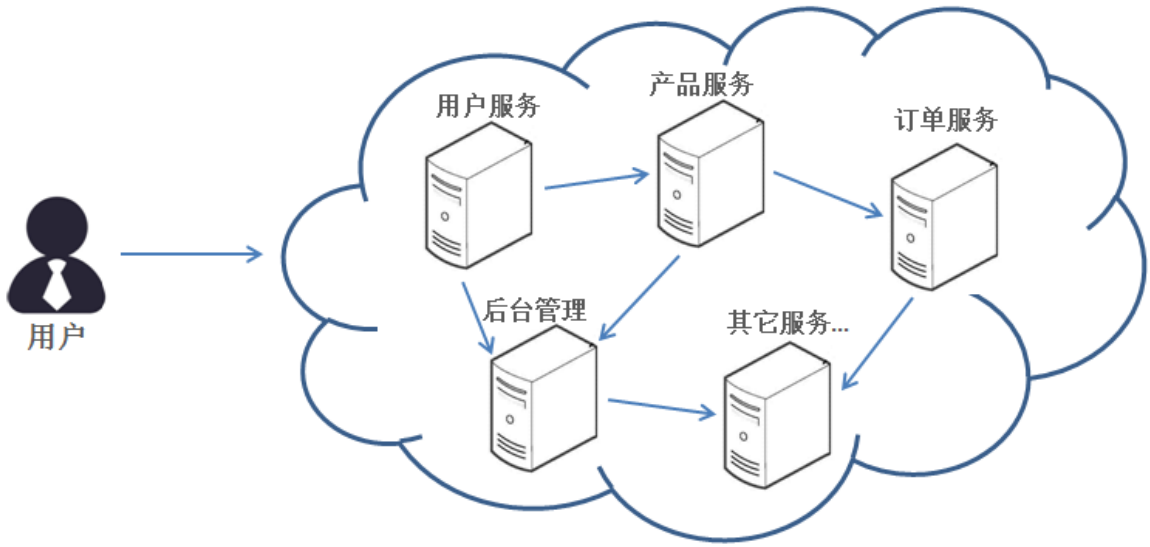


集群结构的好处就是系统扩展非常容易。当系统并发处理能力不够时，只要给集群再增加节点，每个节点部署相同的业务代码就行了。

1.2.分布式

分布式结构就是将一个完整的系统，按照业务功能，拆分成一个个独立的子系统。在分布式结构中，每个子系统都是独立运行在服务器中，它们之间通过RPC或HTTP等方式进行通信。

比如，假设需要开发一个在线商城。按照分布式的思想，我们需要按照功能模块拆分成多个独立的服务，如：用户服务、产品服务、订单服务、后台管理服务、数据分析服务等等。



分布式的好处就是系统之间的耦合度大大降低，可以独立开发、独立部署、独立测试。从而系统更易于扩展，服务的复用性更高。

1.3.集群与分布式的区别

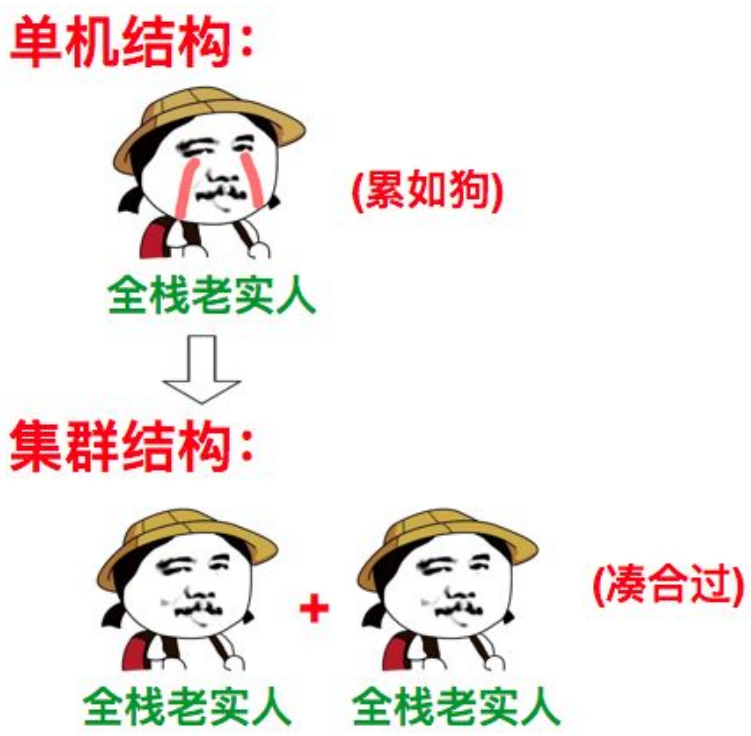
我们可以从多个角度来对集群与分布式进行区别：

1. 从目的性来看：集群解决的是高并发问题，分布式解决的是解耦问题。
2. 从业务部署角度来看：集群是同一个业务部署在多个服务器上，分布式是不同的子业务部署在多个服务器上。
3. 从提升效率的角度来看：集群则是通过提高单位时间内执行的任务数来提升效率的，而分布式是以缩短单个任务的执行时间来提升效率的。

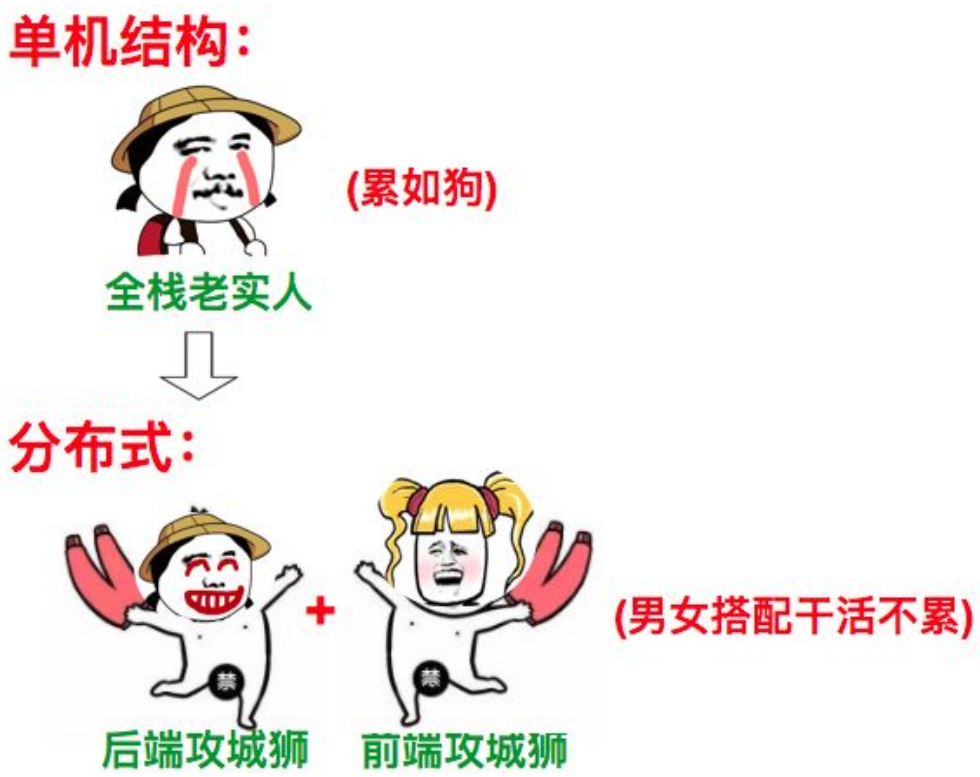
1.4.图解

如果上面的解释还不是很清楚的话，那么请看图：

1.4.1.集群图解



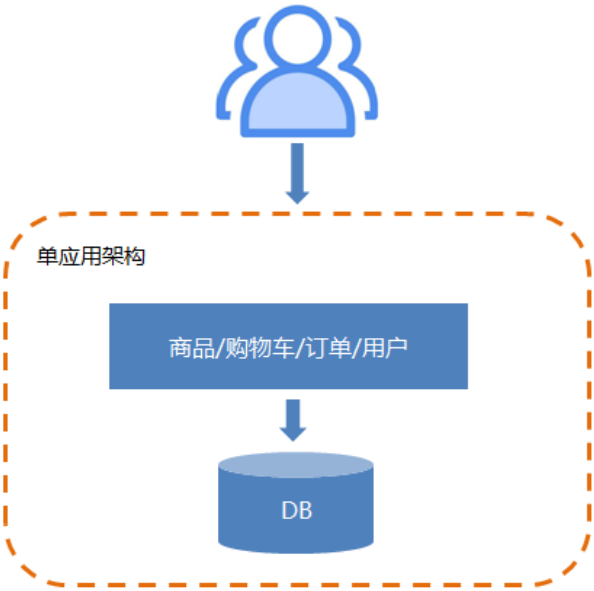
1.4.2.分布式图解



2.软件系统架构演进之路

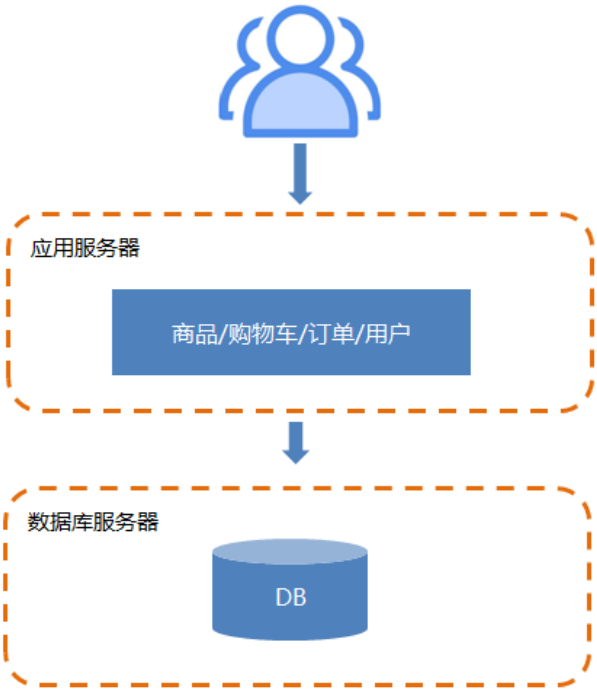
2.1.单应用架构

单应用架构是最早期、最简单的软件架构，简单的说：就是all in one（一应俱全），一台机器完成所有功能。



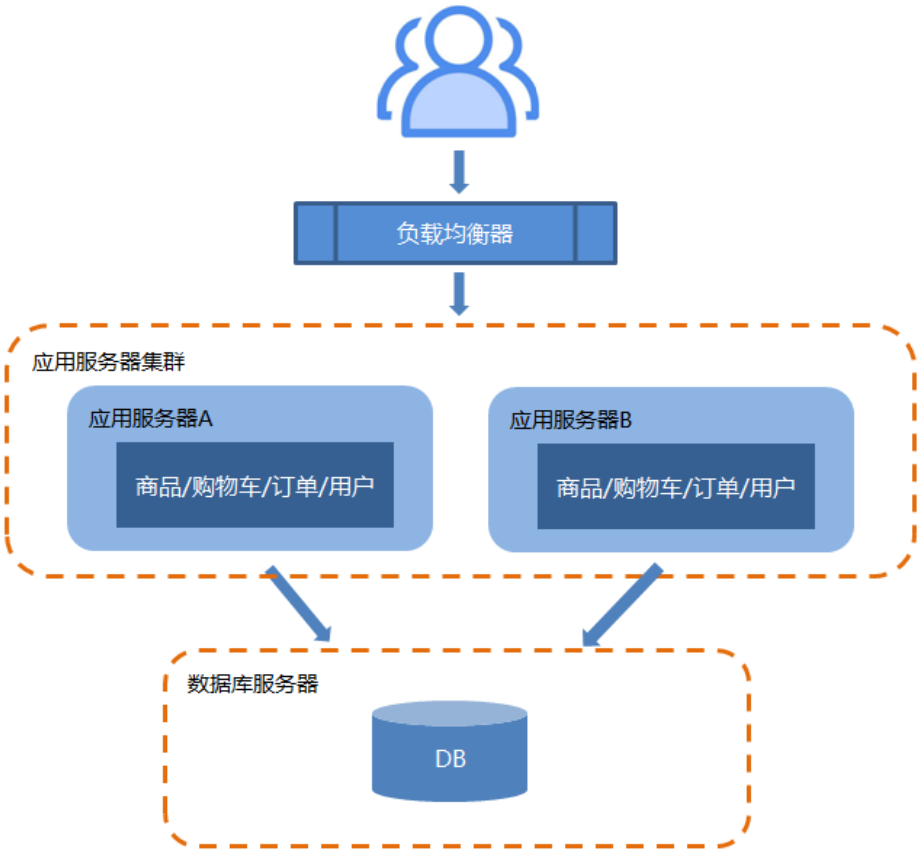
2.2.应用服务器和数据服务器分离

随着业务量的扩大，导致单机负载越来越大。于是我们开始尝试做简单的业务分离，以减轻单机负载压力。也就是将应用服务器和数据库服务器进行分离。



2.3.应用服务器做集群

随着业务量的再次扩大，尤其是并发访问量的急剧增加，受到了单机最大并发请求数的极大限制，这就是高并发问题。为了提高并发访问量，开始尝试对应用服务器做集群。



对于一个集群架构来说，最主要的问题有两个：

- 负载均衡：如何保证请求能够均衡的分配给应用服务器集群中的每一台机器，这就需要使用负载均衡。常见的技术有：Nginx（软件）、F5（硬件）等。

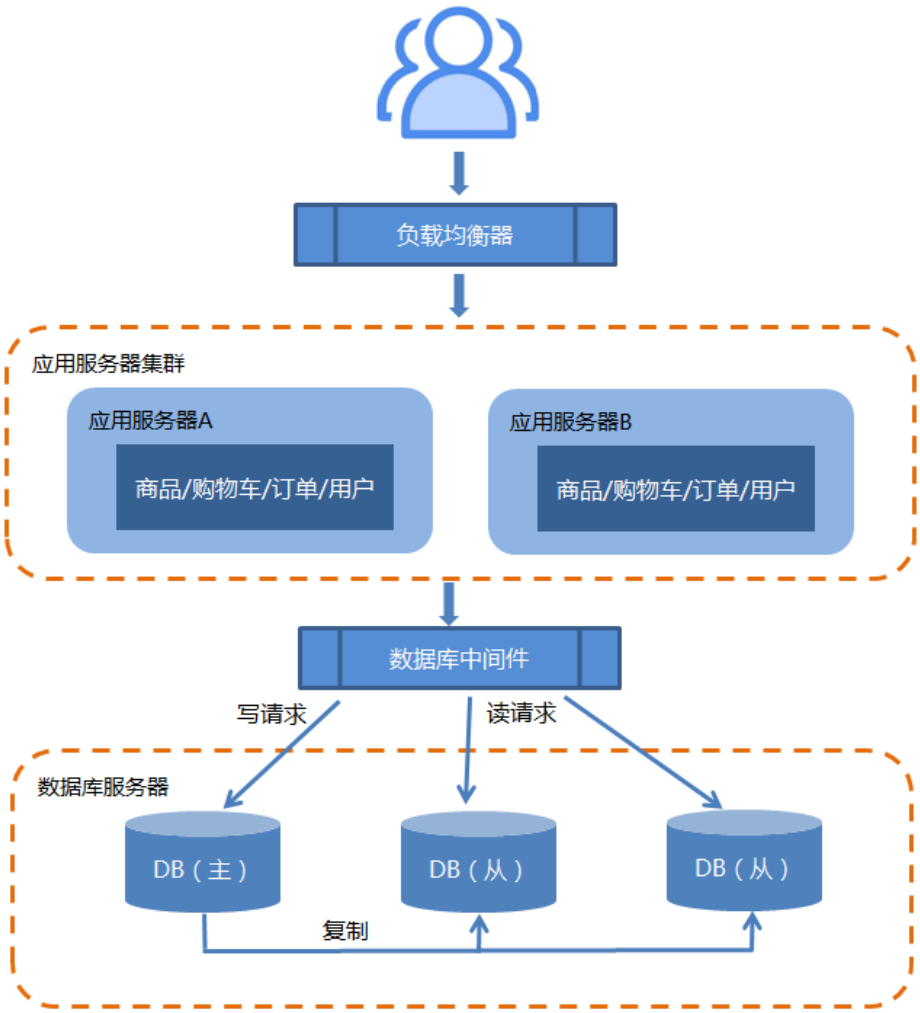
  常用的策略有：轮询、随机、指定权重、ip绑定等。

- session共享：如何保证用户在A服务的状态，能够共享给集群中每一台机器，这就需要使用session共享。常见的技术有：Spring Session等。

  常用的策略有：Session replication、Session sticky

2.4.数据库做集群

应用服务的高并发问题解决了，但是数据库方面也存在高并发问题。所以，又开始尝试搭建数据库集群。



数据库集群搭建与应用服务器集群搭建有所不同：

- 搭建数据库主从集群，实现数据库读写分离，改善数据库负载压力。
- 数据库主机通过复制将数据同步到从机，每台数据库服务器都存储了所有的业务数据。
- 最后，将这些操作封装为一个数据库中间件进行统一管理。常见技术有：MyCat等。MyCat实现了数据源判断问题和分库分表规则问题。

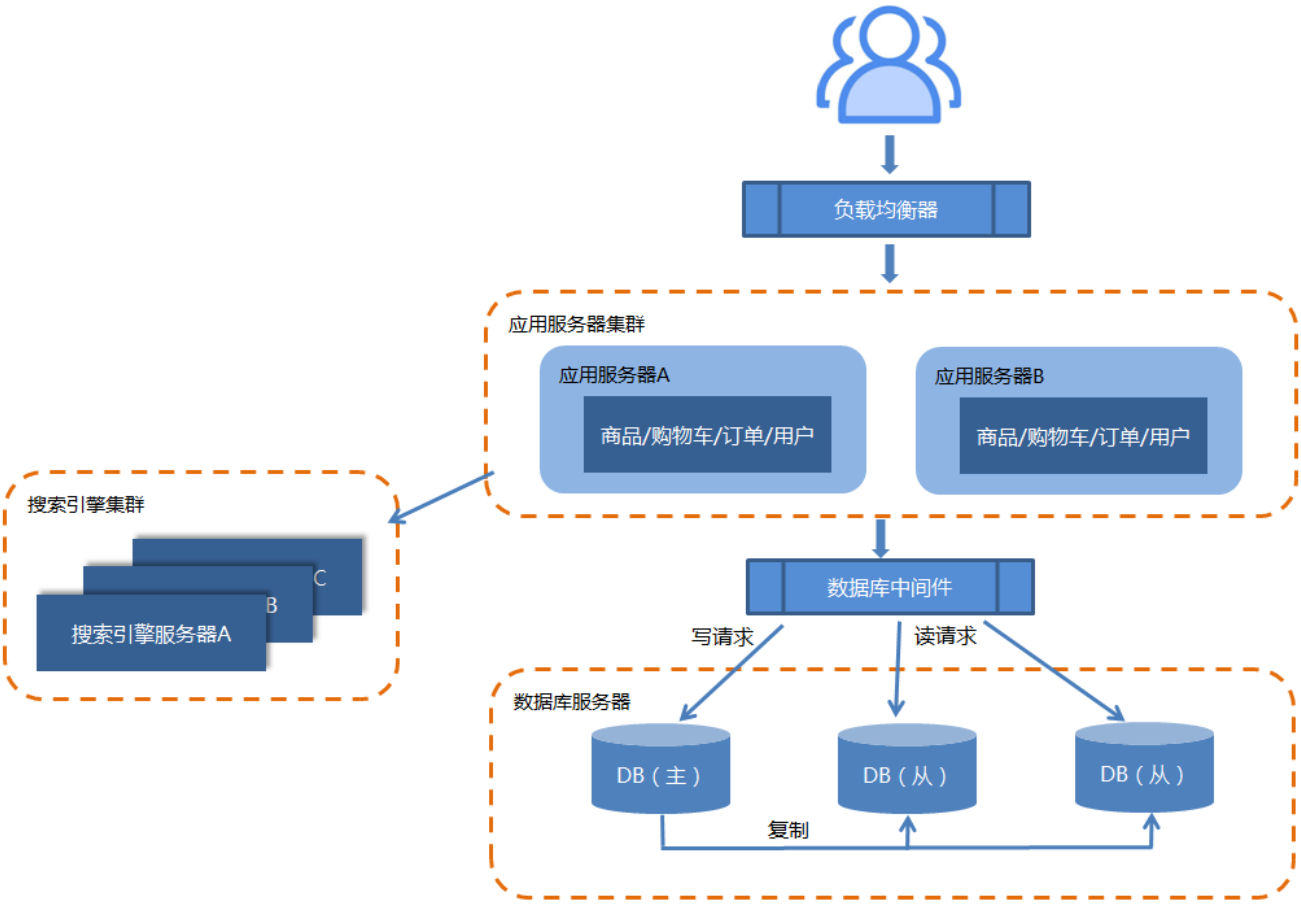
2.5.引入搜索引擎查询

一个系统中，如果没有一个强大的搜索功能将是致命的。

但问题是：搜索功能往往意味着巨大的资源消耗。而且，传统的关系型数据库通过索引来达到快速查询的目的，但是在全文搜索的业务场景下，索引也无能为力，主要体现在：

- 全文搜索的条件可以随意排列组合，如果通过索引来满足，则索引的数量会非常多。
- 全文搜索的模糊匹配方式，索引无法满足，只能用like查询，而like查询是整表扫描，不能实现分词效果、效率非常低。

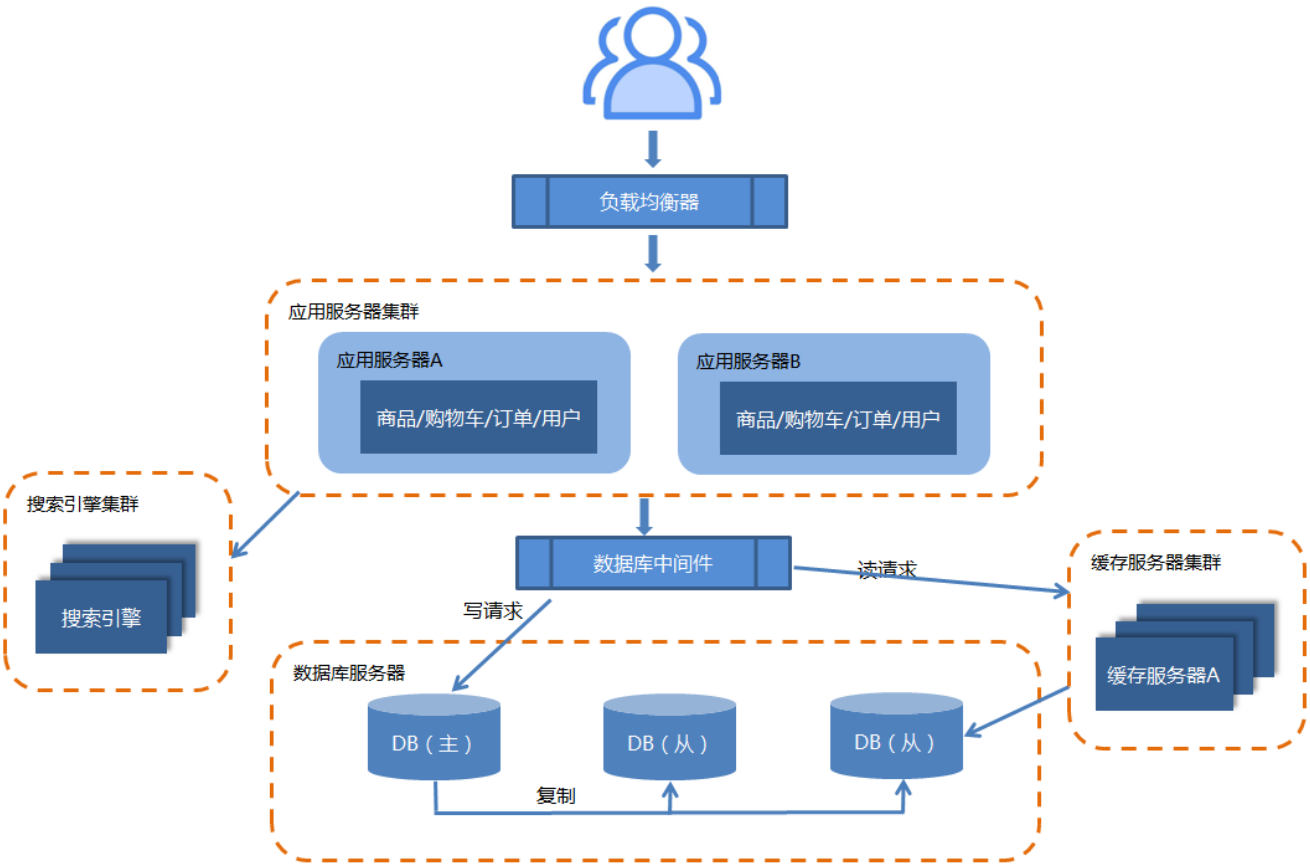
因此，将搜索转移到一个外部的搜索服务器是一个不错的尝试：



目前，搭建搜索引擎服务器常见技术有：Elasticsearch、Solr，它们都是基于Lucene（全文搜索引擎）的。Solr 是传统搜索应用的有力解决方案，但 Elasticsearch 更适用于新兴的实时搜索应用。

2.6.增加缓存

为了增加数据库的QPS（Queries Per Second：每秒查询率），除了做数据库集群外，还可以使用缓存。这样可以将部分读请求分发给缓存服务器，从而降低数据库服务器的负载。

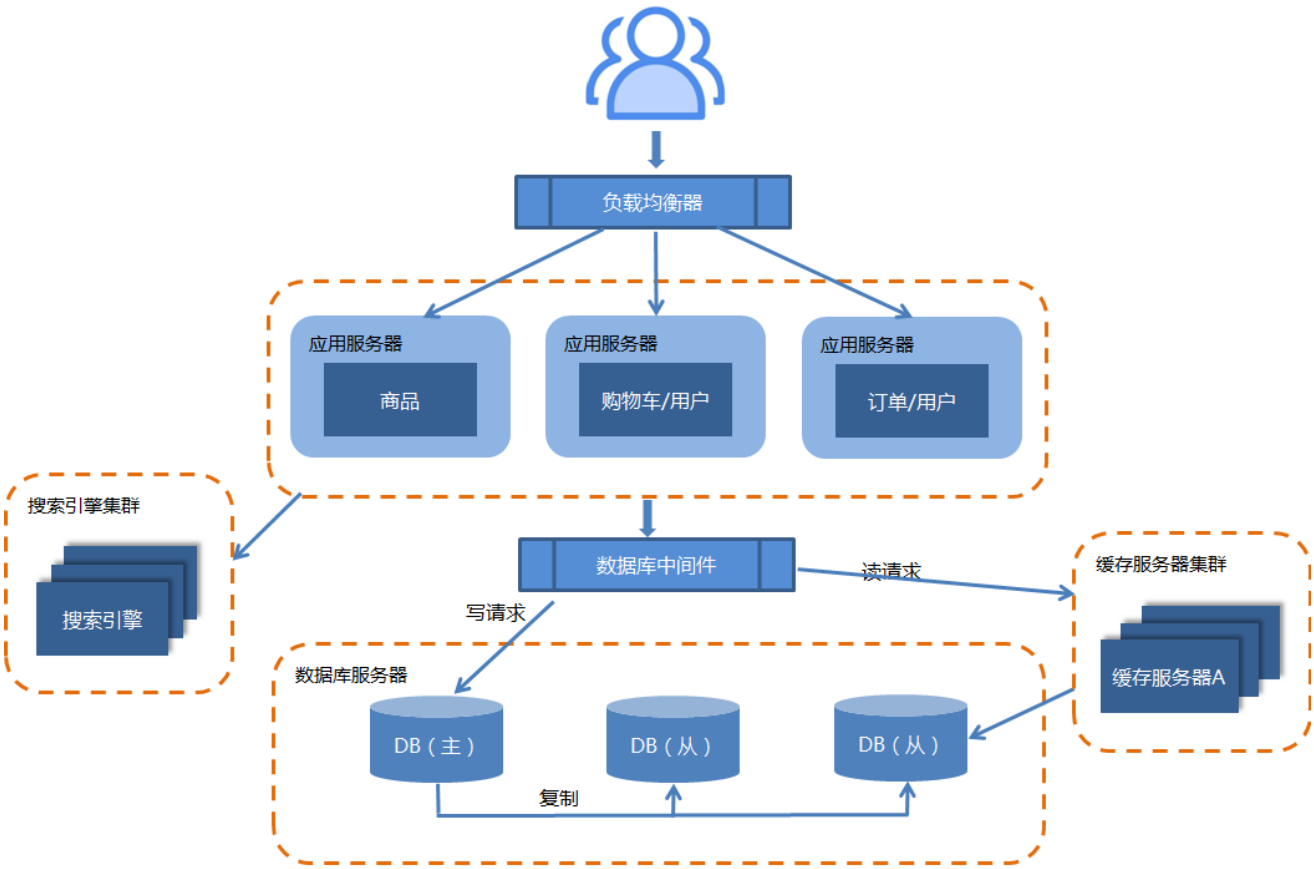


目前常见的缓存技术有：Redia+Memcached（分布式的高速缓存系统）。

2.7.应用拆分为分布式

随着业务的发展，业务越来越多，应用的压力越来越大，工程规模也越来越庞大。这就导致模块与模块之间的呈现紧耦合的关系，使得整个系统变得异常复杂，难以维护和扩展。所以需要对系统进行解耦处理，而且解耦的同时也可以缩短单个任务的执行时间。

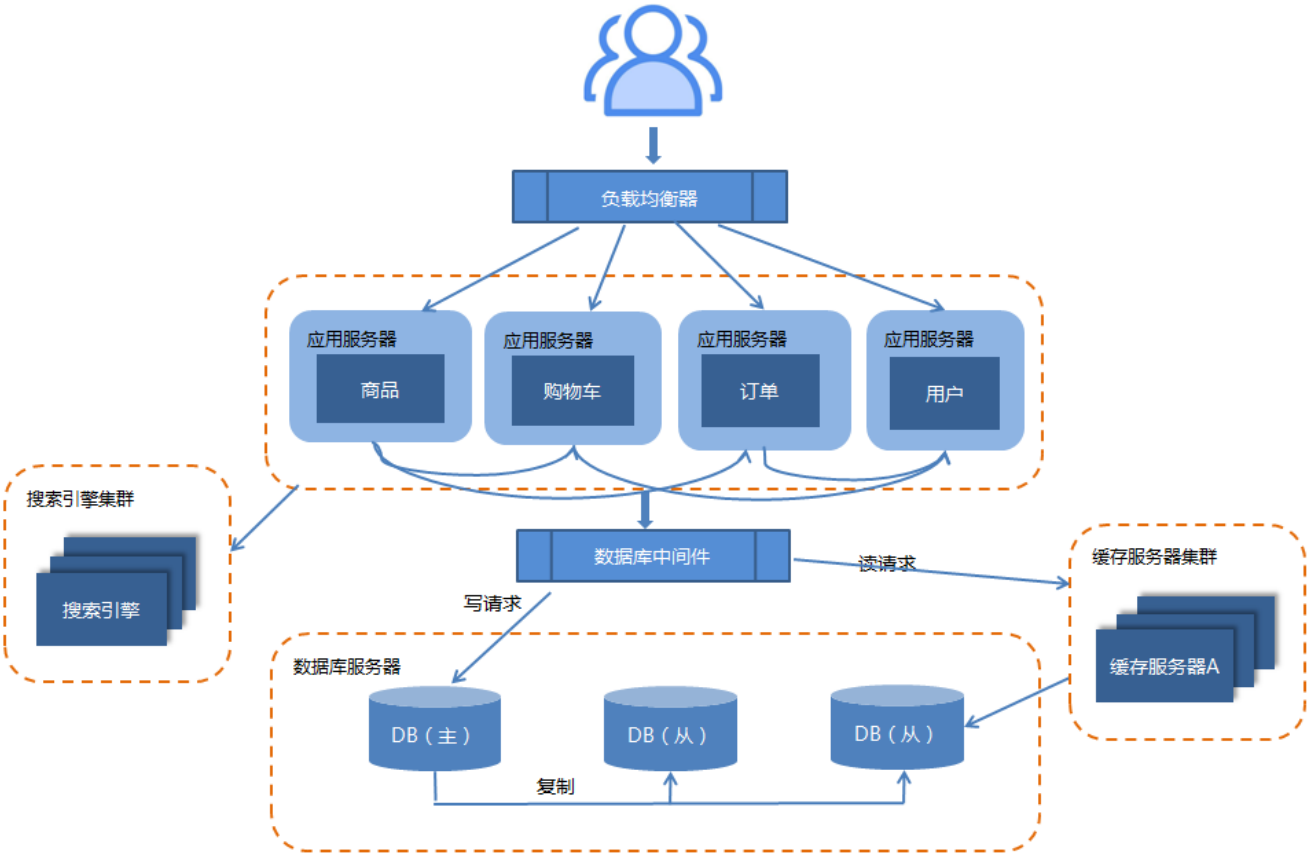
这个时候就可以考虑将应用拆分，也就是将一个系统拆分成多个子系统。而每个子系统完成不同的功能模块，并且独立部署，于是分布式系统架构就出现了。



这样拆分以后，可能会出现一些功能冗余。比如订单模块有对用户数据的查询，购物车模块中也有对用户数据的查询。因此，分布式更多的体现在部署拆分上，而不是功能上的拆分。

2.8.微服务

由于分布式架构在功能拆分上的不彻底，会导致功能冗余，不利于维护和管理等等问题。这就需要我们对功能进行进一步的细化拆分，拆分成一个个微小的服务，这就是微服务了。



微服务就是很小的服务，小到一个服务只对应一个单一的功能，只做一件事。这个服务可以单独部署运行，也可以与其他微服务一起部署。服务之间可以通过RPC或HTTP来相互交互，每个微服务都是由独立的小团队开发，测试，部署，上线，负责它的整个生命周期。

目前常见微服务框架有：Spring Cloud、Dubbo等。

2.9.总结

微服务与分布式的区别：

- 分布式是系统部署方式，更多的体系在物理架构上；微服务是架构设计方式，更多的体系在逻辑架构上；
- 分布式是为了分散系统压力；微服务是为了分散系统能力。
- 分布式架构的子系统是部署在不同的服务器上的；微服务架构中的每一个微服务不一定要部署在不同的服务器上，也可以部署在同一个服务器上。

最后要说的一点是：微服务并非完美的软件系统架构。微服务化后带来的挑战也是显而易见的，例如服务粒度小，数量大，后期运维将会很困难等等。比如：微服务中的事务管理本身就是一个技术难题，目前还没有一种完美的解决方案。

3.分布式架构核心知识

3.1.分布式中的远程调用

分布式架构中的各个服务，都是分别部署在网络上的不同节点中的。那么它们之间当然需要进行远程调用才能互相通信。常见的远程调用技术有：RPC、HTTP ...

- RPC：

  RPC（Remote Procedure Call）一种进程间通信方式。它允许像调用本地服务一样调用远程服务。RPC的目标就是让远程调用更简单和透明。它封装了底层通信的细节。它基于原生TCP通信，速度快，效率高。早期的webservice，现在的dubbo，都是RPC的典型。

- HTTP：

  HTTP其实是一种网络传输协议，基于TCP，规定了数据传输的格式。现在客户端浏览器与服务端通信基本都是采用Http协议。也可以用来进行远程服务调用。使用时采用Restful接口进行调用。

RPC和HTTP的区别：

1. RPC要求服务提供方和服务调用方都需要使用相同的技术；而HTTP无需关注语言的实现，只需要遵循Rest规范即可（跨语言、跨平台）。
2. RPC一般使用TCP协议，效率较高；而HTTP灵活度更高，服务开发迭代会更快（HTTP是应用层协议，TCP是底层协议）。
3. RPC主要用于公司内部的服务调用，性能消耗低，传输效率高，服务治理方便，所以适用于SOA（Service-Oriented Architecture：面向服务架构）。HTTP主要用于对外的异构环境，浏览器接口调用，第三方接口调用等，所以适用于微服务。

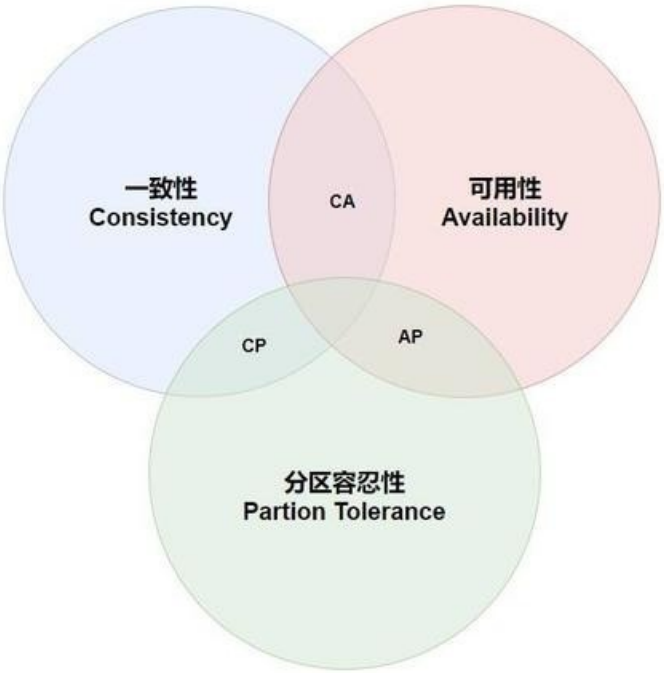
3.2.分布式中的CAP理论

3.2.1.CAP理论简介

> 2000年7月，加州大学伯克利分校的Brewer教授在ACM PODC会议上提出CAP猜想。2年后，麻省理工学院的Seth Gilbert和Nancy Lynch从理论上证明了CAP。之后，CAP理论正式成为分布式计算领域的公认定理。

先了解三个概念：

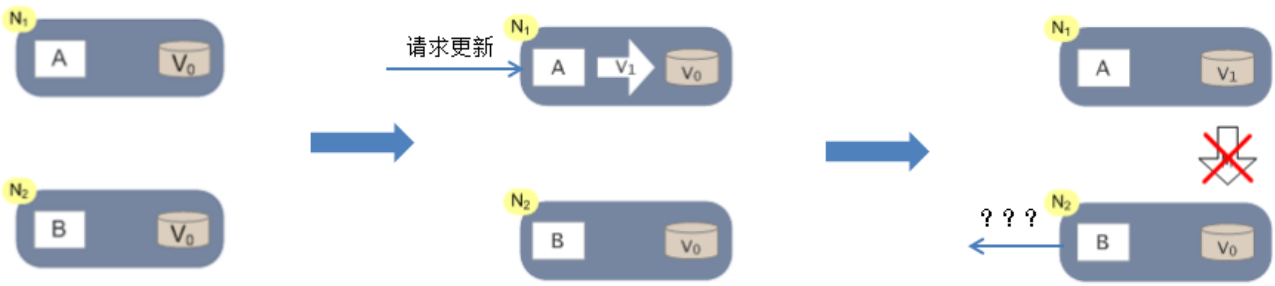
- 一致性（Consistency）：分布式系统在执行过某项请求响应后，所有节点数据仍然处于一致状态。
- 可用性（Availability）：分布式系统中的每一个请求总能够在一定的时间内获得响应（允许响应旧数据）。
- 分区容错性（Partition tolerance）：分布式系统要允许节点之间、或网络分区之间数据传输失败（即数据不一致），为的是保证整个系统的运行。



CAP理论：一个分布式系统最多只能同时满足一致性（Consistency）、可用性（Availability）和分区容错性（Partition tolerance）这三项中的两项。

3.2.2.CAP理论简单证明

这里我们以系统满足P为前提来论述无法同时满足C和A。



1. 步骤1：假设一个分布式系统中有两个节点N1和N2，N1和N2之间可以进行网络通信；N1和N2中都一个应用程序和一个数据库。
2. 步骤2：当N1节点接收一个请求后，将数据更新了。正常情况下，N1和N2节点应该进行数据同步，这样，当对N2节点进行数据请求时，得到的数据是一致的。
3. 步骤3：但是，如果N1和N2之间的网络通信出现故障，那么这两个节点的数据就不能同步了。此时，一致性和可用性就不能同时满足。也就是说：现在只有两种选择：
    - 牺牲数据一致性，响应旧的数据V0给用户；
    - 牺牲可用性，阻塞等待，直到网络连接恢复，数据同步之后，再给用户响应最新数据V1。

3.2.3.CAP权衡

如何在一个分布式系统中进行三选二，权衡三者的关键点取决于业务：

- CA （Consistency + Availability）：关注一致性和可用性（也就是不允许分区容错性）。

它需要非常严格的全体一致的协议。CA 系统不能容忍网络错误或节点错误，一旦出现这样的问题，整个系统就会拒绝写请求，因为它并不知道对面的那个结点是挂掉了，还是因为网络问题。唯一安全的做法就是把自己变成只读的。比如：银行系统。

- CP （consistency + partition tolerance）：关注一致性和分区容忍性（也就是不追求高可用）。

  它关注的是系统里大多数人的一致性协议。这样的系统只需要保证大多数结点数据一致，而少数的结点会在没有同步到最新版本的数据时变成不可用的状态。这样能够提供一部分的可用性。比如：新浪、网易 ...

- AP （availability + partition tolerance）：关注可用性和分区容忍性（也就是不追求数据一致）。

  这样的系统不能达成一致性，需要给出数据冲突，给出数据冲突就需要维护数据版本。AP中一个不错的策略是：保证可用性和分区容错，舍弃一致性，但保证最终一致性。或者说：允许短期内的数据不一致。比如：一些高并发应用：淘宝、12306 ...

最后要说明一点：CA情况在分布式系统中是很少存在。因为在分布式环境下，网络分区是必然的；所以如果舍弃P，意味着要舍弃分布式系统。那也就没有必要再讨论CAP理论了。所以，对于一个分布式系统来说。P是一个基本要求，CAP三者中，只能在CA两者之间做权衡。